

Week14_예습과제_이재린

태그

Ch8. 성능 최적화

8.1 성능 최적화

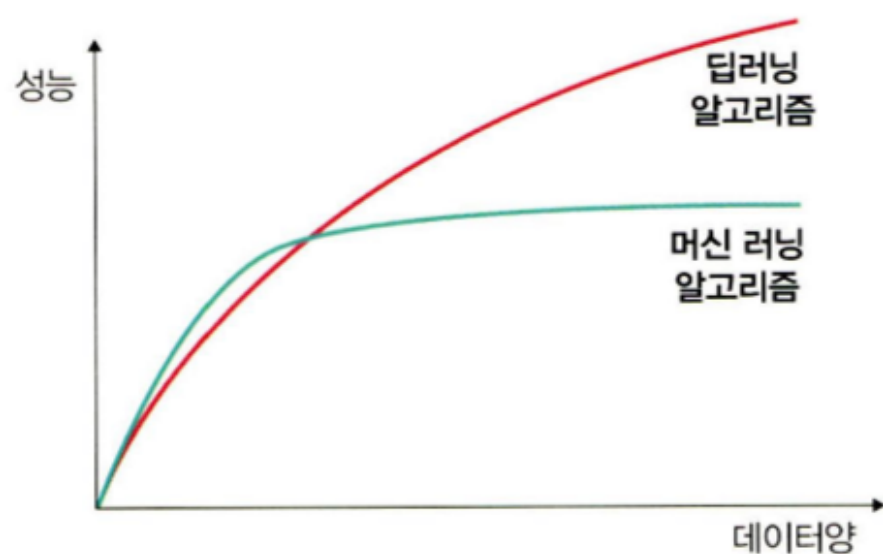
8.1.1 데이터를 사용한 성능 최적화

가장 최선의 방법 >> 많은 데이터

많은 데이터 얻기 어려운 상황에서는..

1. 최대한 많은 데이터 생성

: 딥러닝의 경우 데이터의 양의 성능과의 관계가 두드러짐.



2. 데이터 생성

- 없으면 만들면 됨.

3. 데이터 범위 조정하기

- 활성화 함수 활용 ex) sigmoid $[0,1]$, 하이퍼볼릭 탄젠트 $[-1,1]$

8.1.2 알고리즘을 이용한 성능 최적화

- 최적의 알고리즘을 찾는게 중요

8.1.3 알고리즘 튜닝을 위한 성능 최적화

: 성능 최적화를 할 때 하이퍼파라미터를 변경하면서 훈련시키는 방법이 최적의 성능을 도출시키는 방법이지만 시간 많이 걸림

: 하이퍼파라미터를 잘 선택해야함.

- 진단 : 성능 향상이 멈추었을 때 원인은 여러가지 존재
 - 과적합 : 훈련 성능 > 검증일 때 → 규제화 진행
 - 과소적합 : 둘 다 안 좋을 때 → 네트워크 구조 변경 또는 epoch 조정
 - 변곡점 : 훈련 성능이 검증을 넘어설 때 → 조기 종료
- 가중치 : 초깃값으로 작은 난수 → 비지도 학습으로 사전 훈련 학습
- 학습률 : 네트워크 계층이 많을 때 → 높은 학습률, 네트워크 계층이 적을 때 → 작은 학습률

- 활성화 함수 : if 시그모이드나 하이퍼볼릭 탄젠트, 출력층에서 소프트맥스나 시그모이드 함수 선택
- 배치와 epoch : 일반적으로 큰 에포크 + 작은 배치, but 적절한 배치 크기를 위해 훈련 데이터셋의 크기와 동일하게 하거나 하나의 배치로 훈련
- 옵티마이저 및 손실 함수 : 확률적 경사 하강법, 아담, RMSProp
- 네트워크 구성

8.1.4 앙상블을 이용한 성능 최적화

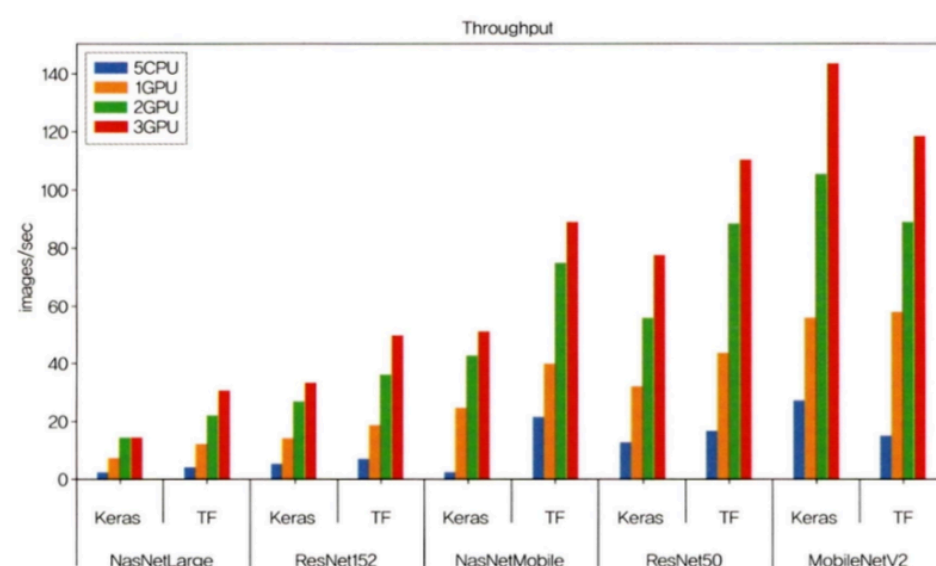
: 두 개 이상의 모델을 섞어서 사용하는 방법

8.2 하드웨어를 이용한 성능 최적화

: 딥러닝 최적화에 왜 HW가 언급되는가? GPU

8.2.1 CPU와 GPU 사용의 차이

♥ 그림 8-3 CPU와 GPU 성능 비교¹



CPU vs GPU

- CPU : ALU(연산) + Control(명령어 해석 및 실행) + Cache(데이터 저장 공간) → 직렬 연산 (한번에 하나의 명령어 처리) good for 행렬연산
- GPU : 병렬 연산 (동시에 여러개의 명령어 병렬적으로 처리) → ALU ⬆ + control + cache ⬇ good for 역전파와 같은 미분 연산

but 개별 코어는 CPU가 더 빠름 → 둘다 분야가 다름

8.2.2 GPU를 이용한 성능 최적화

- 본인의 실행환경 macbookair m1에서는 nvidia의 cuda를 지원하지 않아 대신 applesilliconchip에서 지원하는 병렬연산을 지원하는 mps를 명령어로 활용 가능함.
- GPU만큼 성능이 나오지는 의문임
- 모든 코드에서 gpu명령어 대신 다음과 같은 명령어로 mps를 사용가능

```
if torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")
```

8.3 하이퍼파라미터를 이용한 성능 최적화

Ex) 배치 정규화, 드롭아웃, 조기 종료

8.3.1 배치 정규화를 이용한 성능 최적화

1. 정규화

- 정규화 : 데이터 범위를 사용자가 원하는 범위로 제한하는 것

Ex) image processing 할 때 pixel의 값은 0~255인데 이거 0~1.0으로 범위 바꾸는거

Ex) 아래의 MinMaxScaler()

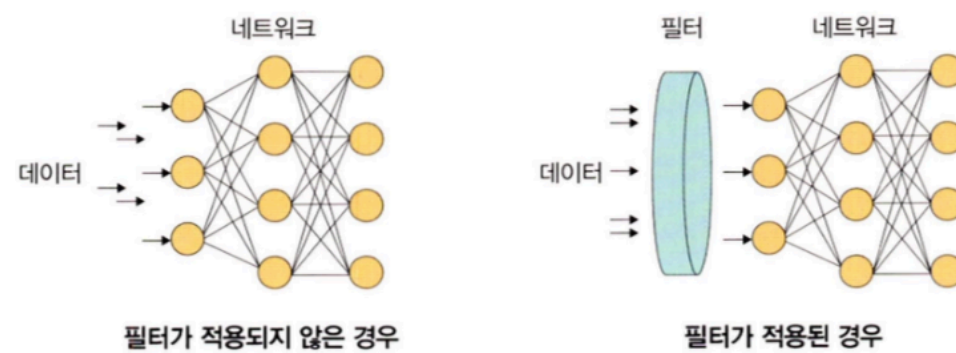
$$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(x : 입력 데이터)

2. 규제화 regularization

- 네트워크에 들어가기 전 모델 복잡도를 줄이기 위해 필터를 적용 → 걸러진 데이터만 투입되어 빠르고 정확한 결과 get 가능

Ex) 드롭아웃, 조기종료

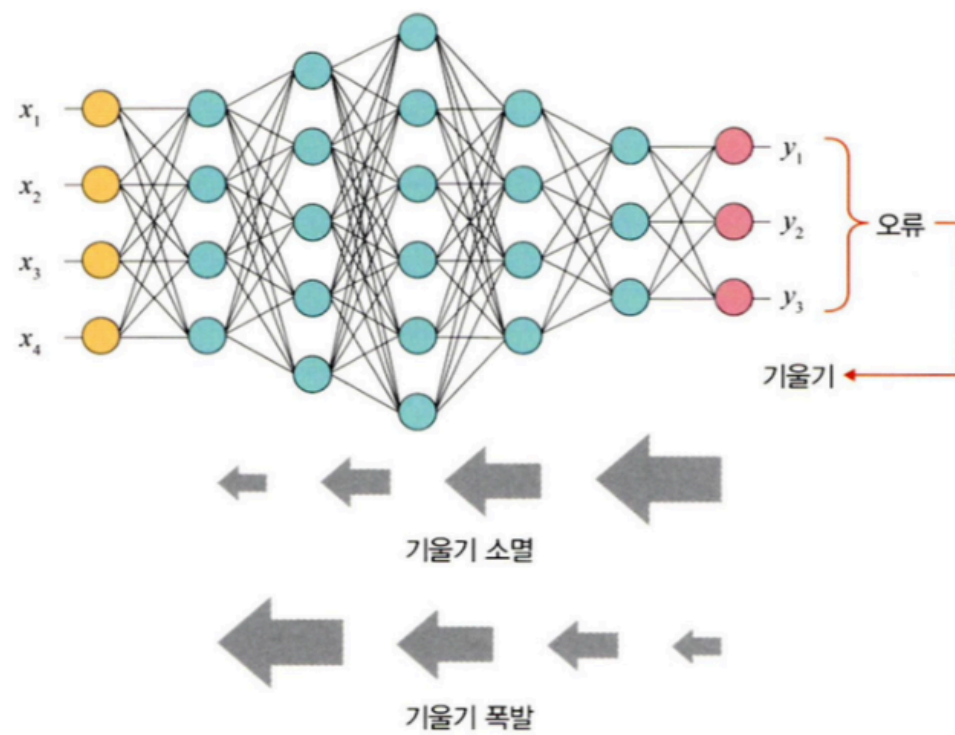


3. 표준화

- Avg=0, std=1의 형태의 데이터로 만드는 것 $(x-m)/sig$

4. 배치 정규화

- 배치 정규화 : 기울기 소멸(오차 정보를 역전파시켜서 기울기가 0에 가까워져 학습 안되는거)와 기울기 폭발(학습에서 기울기가 급격히 커지는 현상)을 해결하기 위한 방법 → ReLU나 초깃값 튜닝, 학습률 등을 조정
 - 은닉층과 입력층 사이에 batch normalization하는 층을 놔둬.



- 기울기 소멸과 폭발의 원인 : nw 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀜(internal covariance shift) → 정규 분포로 만들기 위해 표준화 비스무리한 방식을 미니 배치에 적용해 avg=0, std=1로 유지하게 함

$$\mu\beta \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \text{---①}$$

$$\sigma^2\beta \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 \quad \text{---②}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} \quad \text{---③}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) \quad \text{---④}$$

$$\left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$$

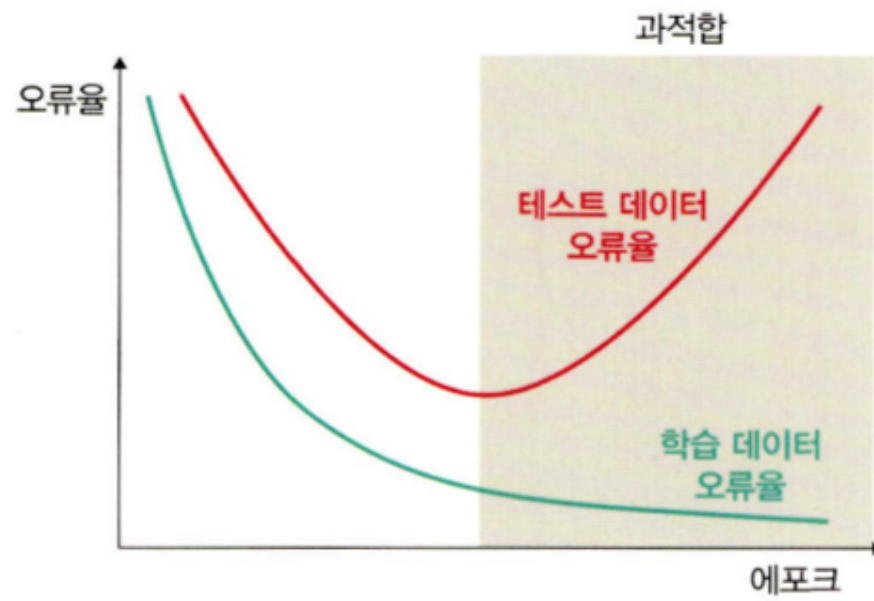
- ① 미니 배치 평균을 구합니다.
- ② 미니 배치의 분산과 표준편차를 구합니다.
- ③ 정규화를 수행합니다.
- ④ 스케일(scale)을 조정(데이터 분포 조정)합니다.

- 단점이 존재하긴 함 ex) 배치 크기가 작을 때 정규화 값이 기존 값과 다른 방향으로 훈련될수도.., RNN의 경우 계층별로 미니 정규화 적용해서 모델이 복잡해짐
- 근데 단점이 있어도 없는 것보다 성능이 나음..쩍

8.3.2 드롭아웃을 이용한 성능 최적화

1. 과적합

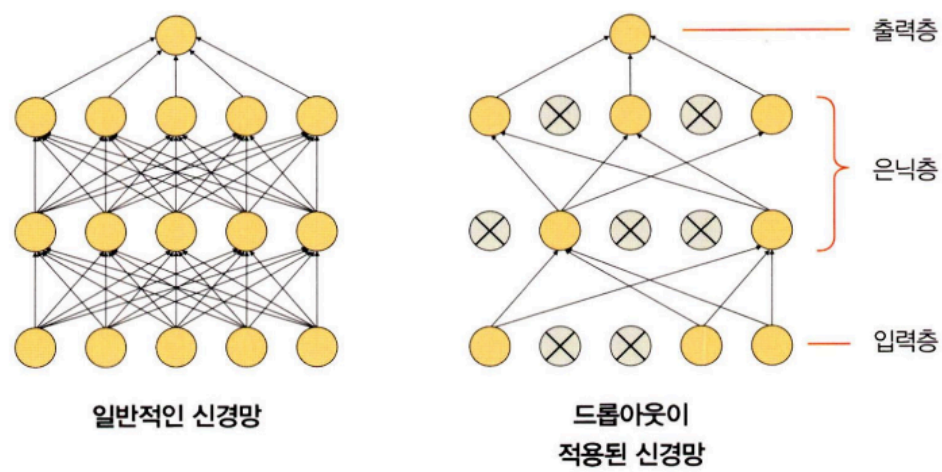
: 훈련 데이터셋에 대해서만 오류가 감소해서 정작 본 게임인 테스트 데이터셋에서 학습을 제대로 못하는거



2. 드롭아웃이란?

: 훈련할 때 일정 비율의 뉴런만 사용하고 나머지는 뉴런의 가중치는 업데이트 하지 않는 방법 → 매 단계마다 뉴런 바꿔줌 (무작위)

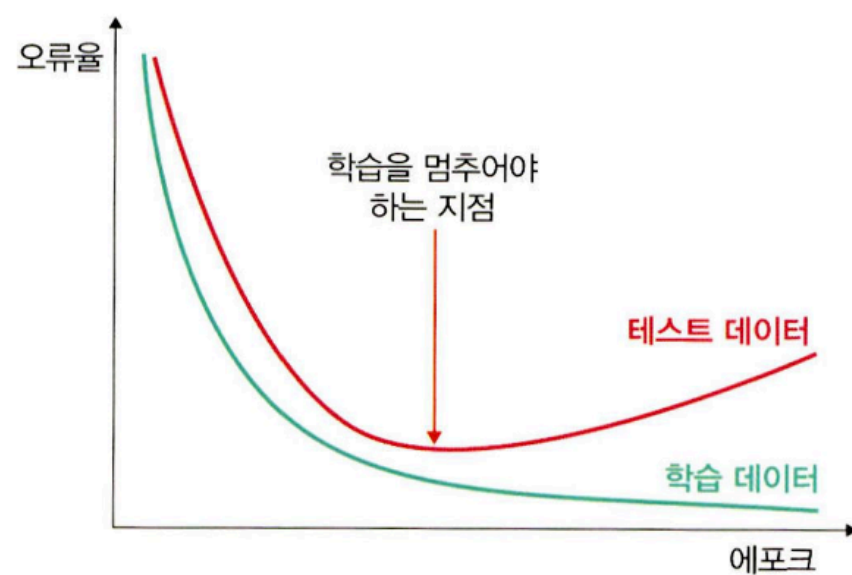
: 테스트 데이터 활용시에는 모든 노드들 사용



*배치 정규화 및 드롭아웃의 유무에 따라 모델의 오차가 줄어드는 형태를 모델의 학습과 이를 시각화했을 때의 그래프로 확인할 수 있다.

8.3.3 조기 종료를 이용한 성능의 최적화

- 과적합을 피하는 규제 기법
- 훈련 데이터와 별도로 검증 데이터 준비하고 각 epoch마다 검증에 대한 오차 측정하여 종료 시점 제어



→ 최고의 성능을 보장하진 않고 종료 시점만 결정함